NTNU
Kunnskap for en bedre verden

Institutt for datateknologi og informatikk

TDT4145 – Datamodellering og databaser

# Prosjektrapport

*Forfattere:*
*Axel Filseth, Leo Egilson Skarbø, Knut Opland Moen*

March 5, 2026

# Contents

**List of Figures** i

**List of Tables** i

**1 ER-diagram** 1

1.1 Antakelser . . . . . 2

1.2 Det ER-modellen ikke uttrykker . . . . . 2

**2 ER-diagram oversatt til relasjonsskjema** 3

2.1 Normalformanalyse . . . . . 4

**3 SQL-script** 5

3.1 Restriksjoner håndtert med triggere . . . . . 10

3.2 Restriksjoner håndtert i applikasjonskode . . . . . 10

**4 Svar på spørsmål** 11

4.1 Hvordan (eller kan) dere sørge for at en instruktør ikke kan være til stede på to forskjellige plasser samtidig? Kan dere få til dette ved hjelp av modellen deres, eller må noe slikt implementeres i programvaren . . . . . 11

4.2 Det samme spørsmålet for en bruker. Kan dette løses i modellen eller må det gjøres i programvaren? . . . . . 11

4.3 Fra hvilket usecase testes det for / opprettes utestengelse (svartelisting)? Det trenger ikke være en av de oppgitte usecasene. . . . . 12

4.4 Statistikk er det noe som må lagres eksplisitt eller går det an å bare gjøre queries for å få svar? Diskuter dette. . . . . 12

# List of Figures

1 ER-diagram. . . . . 1

# List of Tables

# 1 ER-diagram

ER-diagrammet vist i Figure 1 viser entiteter, relasjoner og attributter.

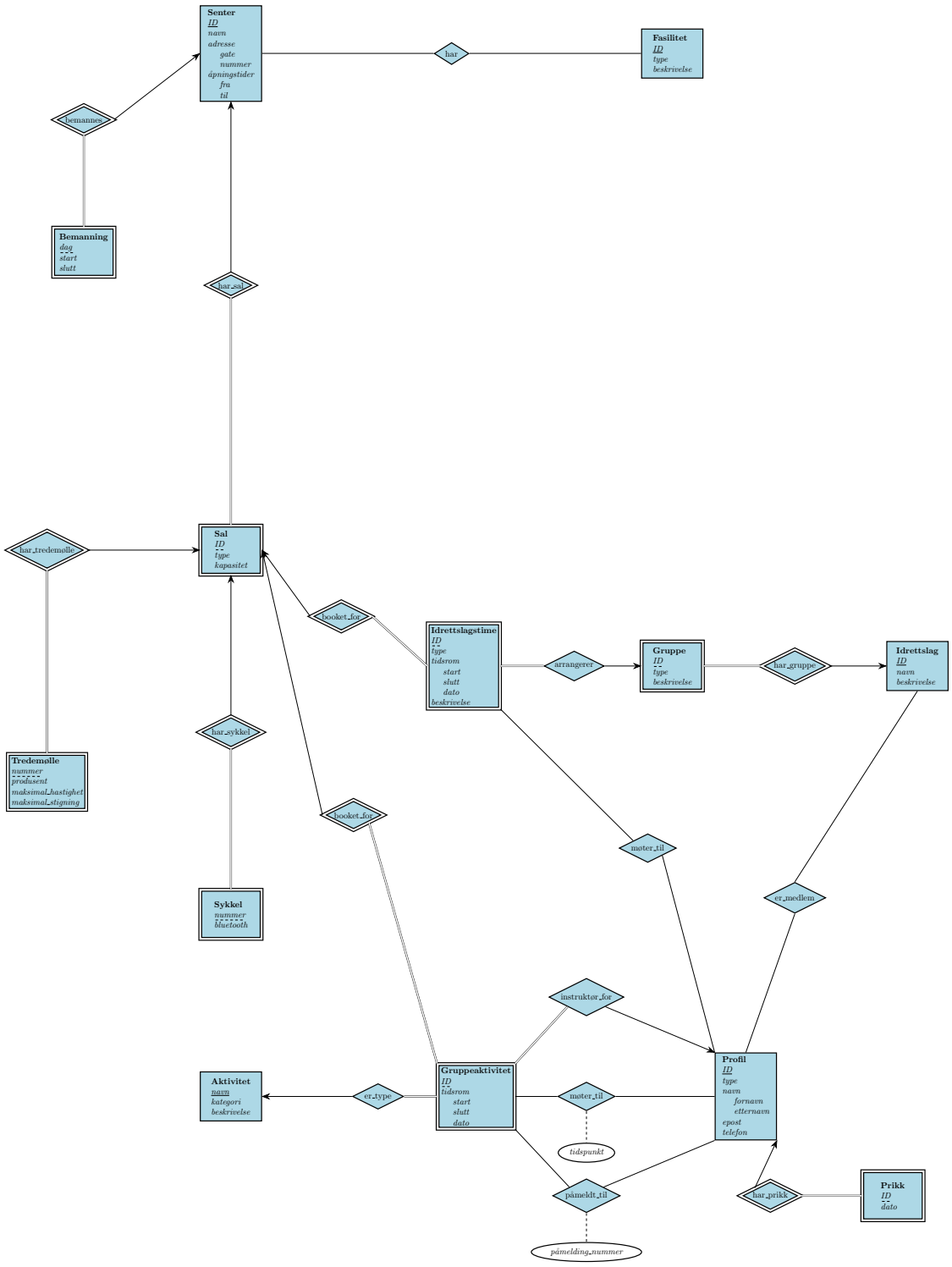


Figure 1: ER-diagram.

## 1.1 Antakelser

Følgende antakelser er gjort:

1. **Fasiliteter.** Vi har én entitet for fasiliteter med *type* som attributt (f.eks. badstue, garderobe, øvelsesrom). Problembeskrivelsen tolkes slik at det viktige er å beskrive hvilke fasiliteter senteret har, ikke attributtene til hver fasilitetstype. En alternativ modellering ville vært å splitte opp i flere entity sets. Deltagelsen fra Fasilitet i *har*-relasjonen er partiell, slik at en fasilitet kan eksistere i systemet uten å være tilknyttet et senter. Dette forenkler vedlikehold: fasiliteter kan opprettes på forhånd og fjernes fra et senter uten å måtte slettes fra databasen.

2. **Bemanning.** Bemanningstidspunktene antas like fra uke til uke; *dag* er derfor brukt som diskriminator (eng, discriminator). Vi tolker at det viktige er *når* senteret er bemannet, ikke hvem som bemanner. Bemanning kunne alternativt vært koblet til Profil. Et tredje alternativ ville vært å ha bemanning som en attributt for senteret. Ulempen med dette er at vi fort kunne fått null-verdier.

3. **Åpningstider.** Vi antar at et treningssenter har samme åpningstider hver dag.

4. **Plasser på aktivitet.** Antall plasser på en aktivitet er definert ut fra kapasiteten til salen aktiviteten foregår i.

5. **Påmelding og kø.** Køsystemet i påmelding tilrettelegges for med et påmeldingsnummer, som fungerer som kølapp. For en aktivitet i en sal med kapasitet $n$ regnes de $n$ profilene med lavest påmeldingsnummer som påmeldt, resten står på venteliste. For idrettslagstime vil vi bruke et dropin-system, der de $n$ første som møter vil få plass.

6. **Sykler og tredemøller.** Utstyr flyttes ikke mellom rom og identifiseres entydig med nummer og sal.

7. **Instruktør.** Vi har generalisert Profil-entiteten, slik at dette fanger både ansatte/instruktører og vanlige medlemmer. Vi skiller mellom rollene med attributten *type*. Dette kan forsvares med at instruktører potensielt også kan være medlemmer, og dermed delta på andre aktiviteter og være med i idrettslag. I praksis vil typen ansatt funke som et ekstra privilegie der profilen kan være instruktør for en aktivitet, men likevel har alle vanlige medlemsfunksjoner.

8. **Idrettslagstime.** For en undergruppe av et idrettslag som arrangerer idrettslagstime, antas det at en profil ikke trenger å være medlem av gruppen for å delta, altså at det holder at profilen er medlem av idrettslaget.

## 1.2 Det ER-modellen ikke uttrykker

Følgende regler og restriksjoner kan brukes i et system som bygger på modellen, men uttrykkes ikke eksplisitt i ER-diagrammet:

1. **Prikk ved for sen møting.** En profil får ikke automatisk prikk dersom den møter under 5 minutter før en aktivitet. For å tilrettelegge for prikksystemet skiller vi mellom *påmeldt_til* og *møter_til*, med tidspunkt som attributt i *møter_til*. Da kan man beregne om en profil skal få prikk ved å sjekke om den er påmeldt til en aktivitet uten å møte opp før 5 minutter før start (inkludert ikke møte opp i det hele tatt). Denne regelen er ikke eksplisitt i modellen.

2. **Utestengelse fra nettbooking.** Ved 3 prikker på 30 dager skal profilen utestenges fra nettbooking. Dette kan regnes ut fra en profils prikker og tilhørende datoer. Modellen viser ikke at profilen da ikke kan bruke nettbooking.

3. **Møting etter aktivitetens slutt.** Det skal ikke være mulig å registrere møting til en aktivitet etter aktivitetens slutt. (Vi antar at det er mulig å møte frem til aktivitetens slutt. Det betyr ikke at profilen unngår prikk dersom den møter under 5 minutter før gruppeaktivitetsstart.) Denne restriksjonen vises ikke i ER-modellen.

4. **Ett tidspunkt, én booking per sal.** For et gitt tidspunkt kan en sal bookes av maksimalt én gruppeaktivitet eller én idrettslagstime. Dette er ikke uttrykt i modellen.
5. **Medlemskap for idrettslagstime.** For at en profil skal kunne møte til en idrettslagstime, må den være medlem av idrettslaget. Dette kravet kommer ikke frem i modellen.

## 2 ER-diagram oversatt til relasjonsskjema

Relasjonsskjemaet under er utledet direkte fra ER-diagrammet ved standardreglene for ER-til-relasjonsoversetting. Primærnøkler er understreket, fremmednøkler er *kursive*, og attributter som er begge deler er *understreket kursive*.

**Fasilitet**(ID, type, beskrivelse)

**Senter**(ID, navn, gate, nummer, fra, til)

**Idrettslag**(ID, navn, beskrivelse)

**Profil**(ID, type, fornavn, etternavn, epost, telefon)

**Aktivitet**(navn, kategori, beskrivelse)

**har_fasilitet**(*senter_ID*, *fasilitet_ID*)

- *senter_ID* → Senter(ID)
- *fasilitet_ID* → Fasilitet(ID)

**Bemanning**(*senter_ID*, dag, start, slutt)

- *senter_ID* → Senter(ID)

**Sal**(*senter_ID*, ID, type, kapasitet)

- *senter_ID* → Senter(ID)

**Tredemølle**(*senter_ID*, *sal_ID*, nummer, produsent, maks_hastighet, maks_stigning)

- (*senter_ID*, *sal_ID*) → Sal(senter_ID, ID)

**Sykkel**(*senter_ID*, *sal_ID*, nummer, bluetooth)

- (*senter_ID*, *sal_ID*) → Sal(senter_ID, ID)

**Gruppe**(*idrettslags_ID*, ID, type, beskrivelse)

- *idrettslags_ID* → Idrettslag(ID)

**Prikk**(*profil_ID*, ID, dato)

- *profil_ID* → Profil(ID)

**er_medlem**(*idrettslags_ID*, *profil_ID*)

- *idrettslags_ID* → Idrettslag(ID)
- *profil_ID* → Profil(ID)

**Idrettslagstime**(*senter_ID*, *sal_ID*, ID, type, start, slutt, dato, beskrivelse, *idrettslags_ID*, *gruppe_ID*)

- (*senter_ID*, *sal_ID*) → Sal(senter_ID, ID)
- *idrettslags_ID* → Idrettslag(ID)
- (*idrettslags_ID*, *gruppe_ID*) → Gruppe(idrettslags_ID, ID)

**møter_til_idrett**(*senter_ID*, *sal_ID*, *idrettslagstime_ID*, *profil_ID*)

- (*senter_ID*, *sal_ID*, *idrettslagstime_ID*) → Idrettslagstime(senter_ID, sal_ID, ID)
- *profil_ID* → Profil(ID)

**Gruppeaktivitet**(*senter_ID*, *sal_ID*, ID, start, slutt, dato, *aktivitet_navn*, *instrukt_ID*)

- (*senter_ID*, *sal_ID*) → Sal(senter_ID, ID)
- *aktivitet_navn* → Aktivitet(navn)
- *instrukt_ID* → Profil(ID)

**møter_til_gruppe**(*senter_ID*, *sal_ID*, *gruppeaktivitet_ID*, *profil_ID*, tidspunkt)

- (*senter_ID*, *sal_ID*, *gruppeaktivitet_ID*) → Gruppeaktivitet(senter_ID, sal_ID, ID)
- *profil_ID* → Profil(ID)

**påmeldt_til**(*senter_ID*, *sal_ID*, *gruppeaktivitet_ID*, *profil_ID*, påmelding_nummer)

- (*senter_ID*, *sal_ID*, *gruppeaktivitet_ID*) → Gruppeaktivitet(senter_ID, sal_ID, ID)
- *profil_ID* → Profil(ID)

## 2.1 Normalformanalyse

Alle tabellene er på **BCNF** (Boyce–Codd normalform). En relasjon er på BCNF dersom venstre side i enhver ikke-triviell funksjonell avhengighet er en supernøkkel. Nedenfor begrunnes dette gruppevis.

**Sterke entiteter – Fasilitet, Senter, Idrettslag, Aktivitet.** Disse har én enkeltattributt som primærnøkkel (*ID*, resp. *navn* for Aktivitet). Alle øvrige attributter bestemmes entydig av primærnøkkelen, og det finnes ingen andre funksjonelle avhengigheter. Dermed er venstre side i enhver ikke-triviell FD en supernøkkel, og tabellene er på BCNF.

**Profil.** Primærnøkkelen er *ID*. I tillegg er *epost* unik for hver profil (brukstilfellene identifiserer brukere ved epostadresse), slik at *epost* → *ID, type, fornavn, etternavn, telefon* også er en ikke-triviell FD. Venstresiden er en kandidatnøkkel og dermed en supernøkkel, så BCNF er oppfylt.

**Svake entiteter – Bemanning, Sal, Tredemølle, Sykkel, Idrettslagstime, Gruppeaktivitet, Gruppe, Prikk.** ER-til-relasjonsoversettingen av svake entiteter gir en sammensatt primærnøkkel bestående av eierens primærnøkkel og diskriminatoren. De eneste ikke-trivielle funksjonelle avhengighetene er av formen primærnøkkel → øvrige attributter, der venstresiden er primærnøkkelen og dermed en supernøkkel. BCNF er derfor oppfylt for alle disse tabellene.

**M:N-relasjoner uten ekstra attributter – har_fasilitet, er_medlem, møter_til_idrett.** Alle attributter inngår i primærnøkkelen. Det finnes ingen ikke-trivielle funksjonelle avhengigheter utover de trivielle, og tabellene er derfor trivielt på BCNF.

**møter_til_gruppe.** Primærnøkkelen er sammensatt av alle fire fremmednøkkelattributtene. *tidspunkt* avhenger funksjonelt av hele primærnøkkelen, og det finnes ingen andre ikke-trivielle FD-er. Tabellen er på BCNF.

**påmeldt_til.** Primærnøkkelen er (*senter_ID, sal_ID, gruppeaktivitet_ID, profil_ID*). *påmelding_nummer* er unikt innenfor én gruppeaktivitet (kølapp-semantikk), slik at (*senter_ID, sal_ID, gruppeaktivitet_ID, påmelding_nummer*) → *profil_ID* også er en ikke-triviell FD. Venstresiden er en kandidatnøkkel og dermed en supernøkkel. Begge ikke-trivielle FD-er har altså en supernøkkel på venstresiden, og tabellen er på BCNF.

# 3 SQL-script

SQL-scriptet under oppretter alle tabellene i databasen med primær- og fremmednøkler, `CHECK`-/`UNIQUE`-restriksjoner samt triggere. Scriptet er skrevet for SQLite.

Listing 1: SQL-script for opprettelse av tabeller.

```sql
-- TDT4145 Prosjektoppgave - SQL-script (oppgave c)
-- Oppretter alle tabeller med primær-/fremmednøkler og restriksjoner.
--
-- Målplattform: SQLite  (PRAGMA foreign_keys = ON må kjøres per tilkobling)
--
-- ==========================================================================
-- Restriksjoner som IKKE kan uttrykkes i SQL og må håndteres i applikasjonskode:
--
--   1. Prikk ved manglende/for sen oppmøte: En profil som er påmeldt en
--      gruppeaktivitet uten å registrere oppmøte senest 5 minutter før start
--      skal få en prikk. Regelen krever tidsberegning utenfor databasen.
--
--   2. Utestengelse (svartelisting): Dersom en profil har >= 3 prikker
--      innenfor de siste 30 dagene, skal profilen utestenges fra elektronisk
--      booking. Applikasjonen må sjekke prikkhistorikk ved booking.
--   3. En aktivitet skal publiseres 48 timer før den holdes. Dette
--      er vanskelig å håndtere med triggere, ettersom det ville krevd at
    aktiviteten
--      ble publisert nøyaktig 48 timer før den holdes.
--
-- Restriksjoner håndhevet med triggere (se nederst i filen):
--
--   3. Oppmøte etter aktivitetens slutt
--   4. Én booking per sal per tidspunkt
--   5. Medlemskap for idrettslagstime
--   6. Avbestillingsfrist (senest 1 time før start)
--   7. Instruktørrolle (kun ansatte kan settes som instruktør)
--
-- ==========================================================================

PRAGMA foreign_keys = ON;

-- ==========================================================================
-- Sterke entiteter
-- ==========================================================================

CREATE TABLE Fasilitet (
    ID          INTEGER PRIMARY KEY AUTOINCREMENT,
    type        TEXT    NOT NULL,
    beskrivelse TEXT
);

CREATE TABLE Senter (
    ID     INTEGER PRIMARY KEY AUTOINCREMENT,
    navn   TEXT    NOT NULL UNIQUE,
    gate   TEXT    NOT NULL,
    nummer TEXT    NOT NULL,
    fra    TIME   NOT NULL,
    til    TIME   NOT NULL
);
```

```sql

CREATE TABLE Idrettslag (
    ID          INTEGER PRIMARY KEY AUTOINCREMENT,
    navn        TEXT    NOT NULL,
    beskrivelse TEXT
);

CREATE TABLE Profil (
    ID        INTEGER PRIMARY KEY AUTOINCREMENT,
    type      TEXT    NOT NULL CHECK (type IN ('medlem', 'ansatt')),
    fornavn   TEXT    NOT NULL,
    etternavn TEXT    NOT NULL,
    epost     TEXT    NOT NULL UNIQUE,
    telefon   TEXT    NOT NULL UNIQUE
);

CREATE TABLE Aktivitet (
    navn        TEXT PRIMARY KEY,
    kategori    TEXT NOT NULL,
    beskrivelse TEXT
);

-- ============================================================================
-- Svake entiteter og M:N-relasjoner
-- ============================================================================

CREATE TABLE har_fasilitet (
    senter_ID    INTEGER NOT NULL,
    fasilitet_ID INTEGER NOT NULL,
    PRIMARY KEY (senter_ID, fasilitet_ID),
    FOREIGN KEY (senter_ID)    REFERENCES Senter(ID),
    FOREIGN KEY (fasilitet_ID) REFERENCES Fasilitet(ID)
);

CREATE TABLE Bemanning (
    senter_ID INTEGER NOT NULL,
    dag       TEXT    NOT NULL CHECK (dag IN (
                  'mandag','tirsdag','onsdag','torsdag',
                  'fredag','lørdag','søndag')),
    start     TIME    NOT NULL,
    slutt     TIME    NOT NULL,
    PRIMARY KEY (senter_ID, dag),
    FOREIGN KEY (senter_ID) REFERENCES Senter(ID),
    CHECK (start < slutt)
);

CREATE TABLE Sal (
    senter_ID INTEGER NOT NULL,
    ID        INTEGER NOT NULL,
    type      TEXT    NOT NULL,
    kapasitet INTEGER NOT NULL CHECK (kapasitet > 0),
    PRIMARY KEY (senter_ID, ID),
    FOREIGN KEY (senter_ID) REFERENCES Senter(ID)
);

CREATE TABLE Tredemølle (
    senter_ID      INTEGER NOT NULL,
    sal_ID         INTEGER NOT NULL,
    nummer         INTEGER NOT NULL,
    produsent      TEXT    NOT NULL,
    maks_hastighet REAL    NOT NULL CHECK (maks_hastighet > 0),
    maks_stigning  REAL    NOT NULL CHECK (maks_stigning  > 0),
    PRIMARY KEY (senter_ID, sal_ID, nummer),
    FOREIGN KEY (senter_ID, sal_ID) REFERENCES Sal(senter_ID, ID)
);

CREATE TABLE Sykkel (
    senter_ID INTEGER NOT NULL,
    sal_ID    INTEGER NOT NULL,
    nummer    INTEGER NOT NULL,
    bluetooth INTEGER NOT NULL DEFAULT 0 CHECK (bluetooth IN (0, 1)),
    PRIMARY KEY (senter_ID, sal_ID, nummer),
    FOREIGN KEY (senter_ID, sal_ID) REFERENCES Sal(senter_ID, ID)
```

```sql
);

CREATE TABLE Gruppe (
    idrettslags_ID INTEGER NOT NULL,
    ID             INTEGER NOT NULL,
    type           TEXT    NOT NULL,
    beskrivelse    TEXT,
    PRIMARY KEY (idrettslags_ID, ID),
    FOREIGN KEY (idrettslags_ID) REFERENCES Idrettslag(ID)
);

CREATE TABLE Prikk (
    profil_ID INTEGER NOT NULL,
    ID        INTEGER NOT NULL,
    dato      DATE    NOT NULL,
    PRIMARY KEY (profil_ID, ID),
    FOREIGN KEY (profil_ID) REFERENCES Profil(ID)
);

CREATE TABLE er_medlem (
    idrettslags_ID INTEGER NOT NULL,
    profil_ID      INTEGER NOT NULL,
    PRIMARY KEY (idrettslags_ID, profil_ID),
    FOREIGN KEY (idrettslags_ID) REFERENCES Idrettslag(ID),
    FOREIGN KEY (profil_ID)      REFERENCES Profil(ID)
);

CREATE TABLE Idrettslagstime (
    senter_ID      INTEGER NOT NULL,
    sal_ID         INTEGER NOT NULL,
    ID             INTEGER NOT NULL,
    type           TEXT    NOT NULL,
    start          TIME    NOT NULL,
    slutt          TIME    NOT NULL,
    dato           DATE    NOT NULL,
    beskrivelse    TEXT,
    idrettslags_ID INTEGER NOT NULL,
    gruppe_ID      INTEGER NOT NULL,
    PRIMARY KEY (senter_ID, sal_ID, ID),
    FOREIGN KEY (senter_ID, sal_ID)            REFERENCES Sal(senter_ID, ID),
    FOREIGN KEY (idrettslags_ID)               REFERENCES Idrettslag(ID),
    FOREIGN KEY (idrettslags_ID, gruppe_ID)    REFERENCES Gruppe(idrettslags_ID, ID
        ),
    CHECK (start < slutt)
);

CREATE TABLE møter_til_idrett (
    senter_ID          INTEGER NOT NULL,
    sal_ID             INTEGER NOT NULL,
    idrettslagstime_ID INTEGER NOT NULL,
    profil_ID          INTEGER NOT NULL,
    PRIMARY KEY (senter_ID, sal_ID, idrettslagstime_ID, profil_ID),
    FOREIGN KEY (senter_ID, sal_ID, idrettslagstime_ID)
        REFERENCES Idrettslagstime(senter_ID, sal_ID, ID),
    FOREIGN KEY (profil_ID) REFERENCES Profil(ID)
);

CREATE TABLE Gruppeaktivitet (
    senter_ID      INTEGER NOT NULL,
    sal_ID         INTEGER NOT NULL,
    ID             INTEGER NOT NULL,
    start          TIME    NOT NULL,
    slutt          TIME    NOT NULL,
    dato           DATE    NOT NULL,
    aktivitet_navn TEXT    NOT NULL,
    instrukt_ID    INTEGER NOT NULL,
    PRIMARY KEY (senter_ID, sal_ID, ID),
    FOREIGN KEY (senter_ID, sal_ID) REFERENCES Sal(senter_ID, ID),
    FOREIGN KEY (aktivitet_navn)    REFERENCES Aktivitet(navn),
    FOREIGN KEY (instrukt_ID)       REFERENCES Profil(ID),
    CHECK (start < slutt)
);
```

```sql
CREATE TABLE møter_til_gruppe (
    senter_ID           INTEGER NOT NULL,
    sal_ID              INTEGER NOT NULL,
    gruppeaktivitet_ID  INTEGER NOT NULL,
    profil_ID           INTEGER NOT NULL,
    tidspunkt           DATETIME NOT NULL,
    PRIMARY KEY (senter_ID, sal_ID, gruppeaktivitet_ID, profil_ID),
    FOREIGN KEY (senter_ID, sal_ID, gruppeaktivitet_ID)
        REFERENCES Gruppeaktivitet(senter_ID, sal_ID, ID),
    FOREIGN KEY (profil_ID) REFERENCES Profil(ID)
);

CREATE TABLE påmeldt_til (
    senter_ID           INTEGER NOT NULL,
    sal_ID              INTEGER NOT NULL,
    gruppeaktivitet_ID  INTEGER NOT NULL,
    profil_ID           INTEGER NOT NULL,
    påmelding_nummer    INTEGER NOT NULL,
    PRIMARY KEY (senter_ID, sal_ID, gruppeaktivitet_ID, profil_ID),
    UNIQUE (senter_ID, sal_ID, gruppeaktivitet_ID, påmelding_nummer),
    FOREIGN KEY (senter_ID, sal_ID, gruppeaktivitet_ID)
        REFERENCES Gruppeaktivitet(senter_ID, sal_ID, ID),
    FOREIGN KEY (profil_ID) REFERENCES Profil(ID)
);

-- ==========================================================================
-- Triggere
-- ==========================================================================

CREATE TRIGGER check_møter_til_gruppe_etter_slutt
BEFORE INSERT ON møter_til_gruppe
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Kan ikke registrere oppmøte etter gruppeaktivitetens slutt.')
    WHERE NEW.tidspunkt > (
        SELECT dato || ' ' || slutt
        FROM Gruppeaktivitet
        WHERE senter_ID = NEW.senter_ID
          AND sal_ID    = NEW.sal_ID
          AND ID        = NEW.gruppeaktivitet_ID
    );
END;

CREATE TRIGGER check_møter_til_idrett_etter_slutt
BEFORE INSERT ON møter_til_idrett
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Kan ikke registrere oppmøte etter idrettslagstime er ferdig.')
    WHERE datetime('now') > (
        SELECT dato || ' ' || slutt
        FROM Idrettslagstime
        WHERE senter_ID = NEW.senter_ID
          AND sal_ID    = NEW.sal_ID
          AND ID        = NEW.idrettslagstime_ID
    );
END;

CREATE TRIGGER check_sal_booking_gruppe_insert
BEFORE INSERT ON Gruppeaktivitet
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Salen er allerede booket av en gruppeaktivitet i dette tidsrommet.')
    WHERE EXISTS (
        SELECT 1 FROM Gruppeaktivitet
        WHERE senter_ID = NEW.senter_ID
          AND sal_ID    = NEW.sal_ID
          AND dato      = NEW.dato
          AND NEW.start < slutt
          AND NEW.slutt > start
    );
```

```
    SELECT RAISE(ABORT, 'Salen er allerede booket av en idrettslagstime i dette tidsrommet.')
    WHERE EXISTS (
        SELECT 1 FROM Idrettslagstime
        WHERE senter_ID = NEW.senter_ID
          AND sal_ID    = NEW.sal_ID
          AND dato      = NEW.dato
          AND NEW.start < slutt
          AND NEW.slutt > start
    );
END;

CREATE TRIGGER check_sal_booking_idrett_insert
BEFORE INSERT ON Idrettslagstime
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Salen er allerede booket av en gruppeaktivitet i dette tidsrommet.')
    WHERE EXISTS (
        SELECT 1 FROM Gruppeaktivitet
        WHERE senter_ID = NEW.senter_ID
          AND sal_ID    = NEW.sal_ID
          AND dato      = NEW.dato
          AND NEW.start < slutt
          AND NEW.slutt > start
    );
    SELECT RAISE(ABORT, 'Salen er allerede booket av en annen idrettslagstime i dette tidsrommet.')
    WHERE EXISTS (
        SELECT 1 FROM Idrettslagstime
        WHERE senter_ID = NEW.senter_ID
          AND sal_ID    = NEW.sal_ID
          AND dato      = NEW.dato
          AND NEW.start < slutt
          AND NEW.slutt > start
    );
END;

CREATE TRIGGER check_medlemskap_idrett_insert
BEFORE INSERT ON møter_til_idrett
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Profil må være medlem av idrettslaget for å møte til idrettslagstime.')
    WHERE NOT EXISTS (
        SELECT 1 FROM er_medlem em
        JOIN Idrettslagstime it ON it.idrettslags_ID = em.idrettslags_ID
        WHERE it.senter_ID = NEW.senter_ID
          AND it.sal_ID    = NEW.sal_ID
          AND it.ID        = NEW.idrettslagstime_ID
          AND em.profil_ID = NEW.profil_ID
    );
END;

CREATE TRIGGER check_medlemskap_idrett_update
BEFORE UPDATE ON møter_til_idrett
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Profil må være medlem av idrettslaget for å møte til idrettslagstime.')
    WHERE NOT EXISTS (
        SELECT 1 FROM er_medlem em
        JOIN Idrettslagstime it ON it.idrettslags_ID = em.idrettslags_ID
        WHERE it.senter_ID = NEW.senter_ID
          AND it.sal_ID    = NEW.sal_ID
          AND it.ID        = NEW.idrettslagstime_ID
          AND em.profil_ID = NEW.profil_ID
    );
END;

CREATE TRIGGER check_avbestillingsfrist
BEFORE DELETE ON påmeldt_til
FOR EACH ROW
```

```sql
BEGIN
    SELECT RAISE(ABORT, 'Avbestilling må skje senest 1 time før aktivitetens start.
        ')
    WHERE datetime('now') > (
        SELECT datetime(dato || ' ' || start, '-1 hour')
        FROM Gruppeaktivitet
        WHERE senter_ID = OLD.senter_ID
          AND sal_ID    = OLD.sal_ID
          AND ID        = OLD.gruppeaktivitet_ID
    );
END;

CREATE TRIGGER check_instruktør_rolle_insert
BEFORE INSERT ON Gruppeaktivitet
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Instruktør må ha type ansatt.')
    WHERE (SELECT type FROM Profil WHERE ID = NEW.instrukt_ID) != 'ansatt';
END;

CREATE TRIGGER check_instruktør_rolle_update
BEFORE UPDATE ON Gruppeaktivitet
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Instruktør må ha type ansatt.')
    WHERE (SELECT type FROM Profil WHERE ID = NEW.instrukt_ID) != 'ansatt';
END;
```

## 3.1 Restriksjoner håndtert med triggere

Følgende restriksjoner krever subspørringer og kan ikke uttrykkes med `CHECK`-restriksjoner i SQLite (SQLite støtter ikke subspørringer i `CHECK`). De håndheves i stedet med `BEFORE INSERT/UPDATE/DELETE`-triggere:

1. **Oppmøte etter aktivitetens slutt.** Det skal ikke være mulig å registrere oppmøte til en aktivitet etter at den er ferdig. Triggerne `check_møter_til_gruppe_etter_slutt` og `check_møter_til_idrett_etter_slutt` avviser slike innsettinger.
2. **Én booking per sal per tidspunkt.** For et gitt tidsrom kan en sal kun ha én gruppeaktivitet eller én idrettslagstime. Fire triggere (insert/update for hvert av de to tabellene) sjekker at nye og oppdaterte aktiviteter ikke overlapper med eksisterende bookinger.
3. **Medlemskap for idrettslagstime.** En profil må være medlem av idrettslaget (via `er_medlem`) for å møte til en idrettslagstime.
4. **Avbestillingsfrist.** Avbestilling av en gruppeaktivitet må skje senest 1 time før aktivitetens start. Triggeren `check_avbestillingsfrist` avviser slettinger fra `påmeldt_til` dersom fristen er utløpt.
5. **Publiseringsfrist.** En gruppeaktivitet kan tidligst legges ut for booking 48 timer før den holdes. Triggeren `check_publiseringsfrist` avviser innsettinger med for lang fremtidshorisont.
6. **Instruktørrolle.** Kun profiler med `type = 'ansatt'` kan settes som instruktør for en gruppeaktivitet.

## 3.2 Restriksjoner håndtert i applikasjonskode

Følgende restriksjoner kan ikke håndheves i databasen og må håndteres av applikasjonsprogrammene:

1. **Prikk ved manglende/for sen oppmøte.** En profil som er påmeldt en gruppeaktivitet uten å registrere oppmøte senest 5 minutter før start, skal få en prikk. Regelen krever tidsberegning og ekstern logikk for å utløse prikken.

2. **Utestengelse (svartelisting).** Dersom en profil har $\geq 3$ prikker innenfor de siste 30 dagene, skal profilen utestenges fra elektronisk booking. Applikasjonen må sjekke prikkhistorikk ved booking.

# 4 Svar på spørsmål

## 4.1 Hvordan (eller kan) dere sørge for at en instruktør ikke kan være til stede på to forskjellige plasser samtidig? Kan dere få til dette ved hjelp av modellen deres, eller må noe slikt implementeres i programvaren

I vår modell lagres instruktøren for en gruppeaktivitet som fremmednøkkelen *instrukt_ID* i *Gruppeaktivitet*-tabellen. Det finnes ingen restriksjon i databaseskjemaet som hindrer at samme *instrukt_ID* opptrer i to gruppeaktiviteter med overlappende tidsrom på samme dato.

En `UNIQUE`-restriksjon kan kun forhindre identiske verdier, ikke overlappende *intervaller*. To aktiviteter for samme instruktør på samme dato med delvis overlappende start-/slutt-tidspunkt ville derfor ikke bli fanget opp.

**Konklusjon.** Denne regelen *kan ikke* uttrykkes i den relasjonelle modellen og må derfor implementeres i applikasjonskoden (eller via en trigger). Før en gruppeaktivitet opprettes eller oppdateres, må applikasjonen sjekke at det ikke allerede finnes en annen aktivitet med samme instruktør der tidsintervallet overlapper.

## 4.2 Det samme spørsmålet for en bruker. Kan dette løses i modellen eller må det gjøres i programvaren?

Resonnementet er det samme som for instruktører, men situasjonen er noe bredere fordi en bruker kan delta i flere typer aktiviteter. En profil kan være registrert i *påmeldt_til* for flere gruppeaktiviteter, og i *møter_til_idrett* for flere idrettslagstimer. Det finnes ingen restriksjon i skjemaet som forhindrer overlappende tidsrom, verken innenfor samme aktivitetstype eller på tvers av de to. En bruker kan altså potensielt kollidere på tre måter:

1. to gruppeaktiviteter som overlapper,
2. to idrettslagstimer som overlapper,
3. en gruppeaktivitet og en idrettslagstime som overlapper.

I alle tilfellene krever overlappsjekken sammenligning av *dato*, *start* og *slutt* på tvers av rader (og potensielt på tvers av tabellene *Gruppeaktivitet* og *Idrettslagstime*) for samme *profil_ID*. Siden tidsattributtene ikke ligger i relasjonstabellene *påmeldt_til* og *møter_til_idrett*, men i *Gruppeaktivitet* og *Idrettslagstime*, kreves det en også en join for å utføre sjekken. Dette kan ikke uttrykkes med `UNIQUE` eller `CHECK`.

**Konklusjon.** Også denne regelen må håndteres i applikasjonskoden. Ved påmelding til en gruppeaktivitet eller oppmøte til en idrettslagstime bør applikasjonen kontrollere at brukeren ikke allerede deltar i en annen aktivitet som overlapper i tid.

## 4.3 Fra hvilket usecase testes det for / opprettes utestengelse (svartelisting)? Det trenger ikke være en av de oppgitte usecasene.

Utestengelse involverer to steg: (1) *oppretting* av prikker som til slutt utløser utestengelsen, og (2) *sjekk* av om en profil er utestengt.

**Oppretting.** Prikker gis til profiler som er påmeldt en gruppeaktivitet uten å registrere oppmøte senest 5 minutter før start. Grunnlaget for dette er **brukstilfelle 3** (registrering av oppmøte): det er oppmøteregistreringene i *møter_til_gruppe* som gjør det mulig å avgjøre hvem som *ikke* har møtt. Selve tildelingen av prikker skjer i et eget brukstilfelle; *tildeling av prikker etter oppmøtefrist*, som etter at oppmøtefristen har passert (5 minutter før start), sammenligner *påmeldt_til* med *møter_til_gruppe* og oppretter en rad i *Prikk* for de som ikke har møtt. Når en profil har fått $\geq 3$ prikker innenfor de siste 30 dagene, er den utestengt.

**Sjekk.** Utestengelsen bør testes for i **brukstilfelle 2** (booking). Før en booking godkjennes, sjekker applikasjonen antall prikker for profilen innenfor de siste 30 dagene. Dersom det er $\geq 3$, avvises bookingen.

Brukstilfelle 6 demonstrerer svartelistingsmekanismen eksplisitt, men den naturlige utløseren i et produksjonssystem er brukstilfellet for prikk-tildeling beskrevet over, kombinert med sjekken i bookingsteget.

## 4.4 Statistikk er det noe som må lagres eksplisitt eller går det an å bare gjøre queries for å få svar? Diskuter dette.

De aller fleste statistikkene som SiT etterspør kan utledes med vanlige SQL-spørringer mot den normaliserte databasen, uten å lagre aggregerte tall eksplisitt.

**Eksempler på statistikk som kan beregnes med queries.**

- **Antall påmeldte per økt:** `COUNT(*)` på *påmeldt_til* gruppert på gruppeaktivitet.
- **Hvem som deltok på en gitt trening:** spørring mot *møter_til_gruppe*.
- **Personen(e) med flest gruppetimer i en måned** (brukstilfelle 7): `COUNT(*)` på *møter_til_gruppe* gruppert på *profil_ID*, filtrert på måned.
- **Par som trener sammen** (brukstilfelle 8): selvjoin på *møter_til_gruppe*.
- **Ukeplan** (brukstilfelle 4): spørring mot *Gruppeaktivitet* filtrert på datoperiode.
- **Prikkstatus og utestengelse:** beregnes fra *Prikk*-tabellen ved å telle prikker innenfor de siste 30 dagene.

**Tilfeller der eksplisitt lagring kan være nødvendig.** Problemet oppstår når underliggende data endrer seg over tid og man trenger historiske toppverdier. Det viktigste eksempelet er *det maksimale antallet påmeldte per økt*. Dersom en bruker melder seg av en økt og raden slettes fra *påmeldt_til*, mister man informasjon om at det på et tidspunkt var flere påmeldte. Med kun gjeldende data kan man finne nåværende antall påmeldte, men ikke det historisk høyeste antallet.

Merk at *påmeldt_til* kan inneholde både bekreftede plasser og ventelisteplasser (rader der `påmelding_nummer` > *Sal.kapasitet*). Avhengig av hva man ønsker å måle, totalt antall registrerte eller kun bekreftede plasser, må spørringen filtrere tilsvarende.

For å bevare den historiske toppverdien finnes flere alternativer:

1. Legge til et felt `maks_påmeldte` på *Gruppeaktivitet* som oppdateres ved hver ny påmelding. I SQLite kan dette vedlikeholdes automatisk med en `AFTER INSERT`-trigger på *påmeldt_til* som teller antall rader for den aktuelle gruppeaktiviteten og oppdaterer feltet dersom det nye antallet overstiger gjeldende verdi.

2. Beholde avmeldte rader med en statuskolonne (*soft delete*) i stedet for å slette dem, slik at man alltid kan telle det totale antallet som har vært påmeldt. Ulempen er at alle andre spørringer mot *påmeldt_til* (f.eks. kapasitetssjekk ved booking) da må filtrere på status, noe som øker kompleksiteten i applikasjonskoden.

3. Bruke `påmelding_nummer` som tilnærming: siden nummeret tildeles sekvensielt, gir `MAX(påmelding_nummer)` blant gjenværende rader et estimat, men verdien blir upresis ved avmeldinger, spesielt dersom personen (eller personene) med høyest nummer melder seg av.

**Konklusjon.** Statistikk trenger i hovedsak *ikke* lagres eksplisitt i vår modell, det meste kan utledes av de normaliserte tabellene med vanlige SQL-spørringer. Unntaket er historiske toppverdier, som det maksimale antallet påmeldte per økt, der man enten må lagre verdien eksplisitt eller benytte en "myk-slette-strategi" ("soft delete") for å unngå tap av informasjon.